

Web Programming Lab

JavaScript Asynchronous Programming

Week 8



Hello, World! (Again)

```
<!-- index.html -->

<!DOCTYPE html>

<body>
  <h1>Nothing here</h1>
</body>
<script src="./script.js"></script>
```

```
// script.js

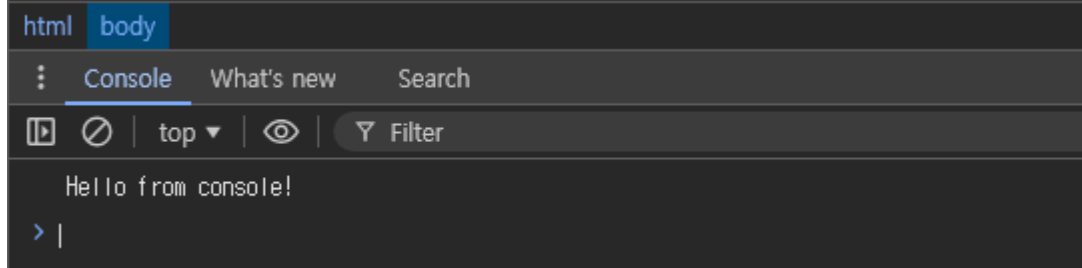
"use strict";

alert("Hello from alert!");
console.log("Hello from console!");
```

127.0.0.1:5500 내용:

Hello from alert!

확인



JavaScript (ES6)
[known limitations](#)

```
1 // script.js
2
3 "use strict";
4
5 → console.log("Hello from console!");
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>

Done running (2 steps)

Print output (drag lower right corner to resize)

Hello from console!

Frames

Objects



Symbol.iterator

```
"use strict";

// range: iterable
let range = { from: 3, to: 7 };
range[Symbol.iterator] = function() {
  return {
    current: this.from,
    last: this.to,
    next: function() {
      if (this.current < this.last) {
        return { done: false, value:
this.current++ };
      } else {
        return { done: true };
      }
    }
  };
};
```

```
// -> continued from previous box

// iter: iterator
let iter = range[Symbol.iterator]();

console.log(iter.next());
console.log(iter.next());

for (let i of range) {
  console.log(i);
}

console.log(range);
console.log(iter);
```



Array-like

```
"use strict";

// arrlike: array-like
let arrlike = {
  0: 123,
  1: 456,
  length: 2,
};

// arr: array (which is iterable)
let arr = Array.from(arrlike);

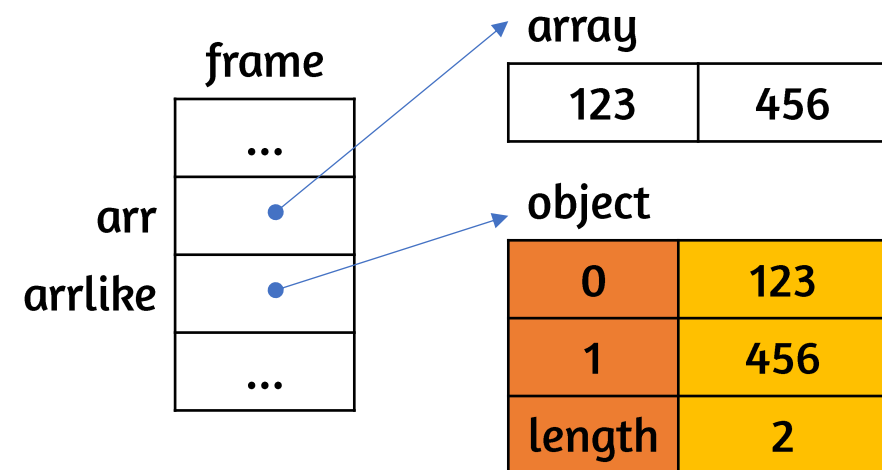
arr.push(789);

console.log(arr.pop());
```

```
// -> continued from previous box

for (let i of arr) {
  console.log(i);
}

// for (let i of arrlike) {
//   console.log(i);
// }
```





Practice: Iterator

```
"use strict";

function Iterable(from, to, step) {
  this[Symbol.iterator] = function() {
    return {
      // fill here
    };
  };
  return this;
}

let iterable = new Iterable(1, 100, 11);

for (let i of iterable) {
  console.log(i);
}
```

result:

1
12
23
34
45
56
67
78
89

spoiler alert: solution in next slide



Solution: Iterator

```
function Iterable(from, to, step) {  
  this[Symbol.iterator] = function() {  
    return {  
      current: from,  
      last: to,  
      next: function() {  
        if (this.current < this.last) {  
          let result = { done: false, value: this.current };  
          this.current += step;  
          return result;  
        } else {  
          return { done: true };  
        }  
      }  
    };  
  };  
  return this;  
}
```



.forEach()

```
"use strict";

let arr = [-1, 2, 3, -4, 5];
let res = [];

// create new result array
arr.forEach(num => { res.push(Math.abs(num)); });
console.log(arr);
console.log(res);

// or just do it in-place
res.forEach((num, idx, array) => { array[idx] = num * 2; });
console.log(res);
```



.every() and .some()

```
const onlyint = [10, 44, -3, 0, 5];
const mixed = [1, 5, 7, "8"];

console.log(onlyint.every(element => { return Number.isInteger(element); }));
console.log(mixed.every(element => { return Number.isInteger(element); }));

console.log(mixed.some(element => { return Number.isInteger(element); }));

// or...
console.log(onlyint.every(element => Number.isInteger(element)));
```

```
const arr = [0, 8, 6, 9, 100, 85];

console.log(arr.every((num, idx) => (idx == 0 || num % idx == 0)));
```

🌀 .map(), .filter(), and .reduce()

```
const arr = [1, 2, 4, 5, 7];

const res1 = arr.map(num => num * 2);
console.log(res1);

const res2 = arr.filter(num => num % 2 == 0);
console.log(res2);

const res3 = arr.reduce((acc, num) => acc + num, 0);
console.log(res3);
```



Timer

```
const func1 = (aa, bb) => {  
  console.log(aa + bb);  
}  
  
const func2 = (cc) => {  
  console.log(aa + bb);  
}  
  
let a = 100;  
let b = 200;  
let c = 300;  
  
setTimeout(func1, 1000, a, b);  
setInterval(func2, 1000, c);
```



Practice: Timer

```
function Timer(name) {  
  let cnt = 0;  
  
  const myFunction = () => {  
    console.log(name + ": " + cnt);  
    cnt++;  
    if (cnt > 3) {  
      clearInterval(id);  
    }  
  }  
  
  const id = setInterval(myFunction, 1000);  
}  
  
setTimeout(() => { let t1 = new Timer("t1"); }, 0);  
setTimeout(() => { let t2 = new Timer("t2"); }, 500);  
  
// what will be printed?
```



Promise

```
const promise = new Promise((resolve, reject) => {  
  const r = Math.random();  
  setTimeout(() => {  
    if (r < 0.7) { resolve(`Resolved with ${r}`); }  
    else { reject(`Rejected with ${r}`); }  
  }, 1000);  
});  
  
promise  
  .then((result) => { console.log(result); })  
  .catch((error) => { console.log(error); });
```



Promise Chaining

```
function openDB(db) {  
  return new Promise((resolve, reject) => {  
    const r = Math.random();  
    setTimeout(() => {  
      if (r < 0.8) { resolve(`opened ${db}`); }  
      else { reject(new Error("Cannot open DB")); }  
    }, 500);  
  });  
}  
  
function getUser(user) {  
  return new Promise((resolve, reject) => { /* ... */ }, 1000);  
}  
  
openDB("myDB")  
  .then((res) => { console.log(res); return getUser("myuser"); })  
  .then((res) => console.log(res))  
  .catch((err) => console.log(err));
```



Async/Await

```
const url = "https://jsonplaceholder.typicode.com/todos/1"

// using promise chain
fetch(url)
  .then(response => response.json())
  .then(json => console.log(json))
  .catch(err => console.log(err));

// using async/await
async function func() {
  try {
    const response = await fetch(url);
    const json = await response.json();
    console.log(json);
  } catch {
    console.log(err);
  }
  console.log("Hello from func1");
}
```